

RRC-Raptor-V4H

- Ethernet

V0.0.1

Contents

i.	VERSION LIST	5
ii.	HASHTAG	6
1.	Overview.....	7
2.	Ethernet.....	8
3.	How to Run UMSD_MCLI.....	11
3.1.	Run with an Interactive Interface.....	15
3.2.	Run with the Configuration File.....	18
4.	How to Use UMSD_MCLI.....	22
5.	REFERANCE	24

LIST OF FIGURES

FIGURE 1. ETHERNET BLOCK DIAGRAM	8
FIGURE 2. THE POSITION OF EACH ETHERNET CONNECTOR	8

LIST OF TABLES

TABLE 1. DEFAULT BOOTSTRAP SETTINGS ON SW18 – SW21 FOR 88Q6113	12
TABLE 2. DEFAULT BOOTSTRAP SETTINGS ON SW22 AND SW23 FOR 88Q5040	13
TABLE 3. DEFAULT BOOTSTRAP SETTINGS ON SW24 AND SW25 FOR 88Q5040	14

i. **VERSION LIST**

Description	Version	Date	Author
Ethernet & UMSD_MCLI	V0.0.1	2024/06/26	HowardChao

ii. HASHTAG

#V4H

#BSP

#ubuntu

#Ethernet

#UMSD_MCLI

#Marvell Ethernet Switch

1. Overview

Target : Introduce how to test Ethernet function of each port and use
UMSD_MCLI with Marvell Ethernet Switch on V4H-Raptor.

2. Ethernet

Ethernet Block Diagram[1]

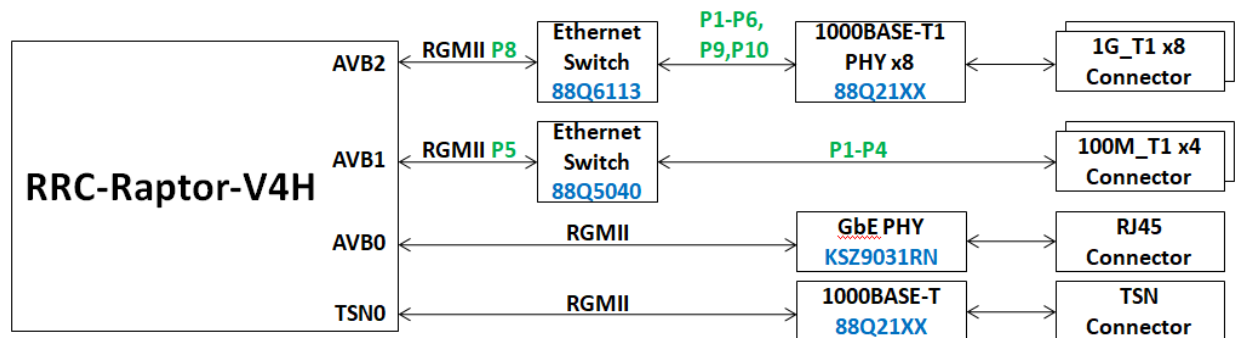


Figure 1. Ethernet Block Diagram

The position of each Ethernet connector

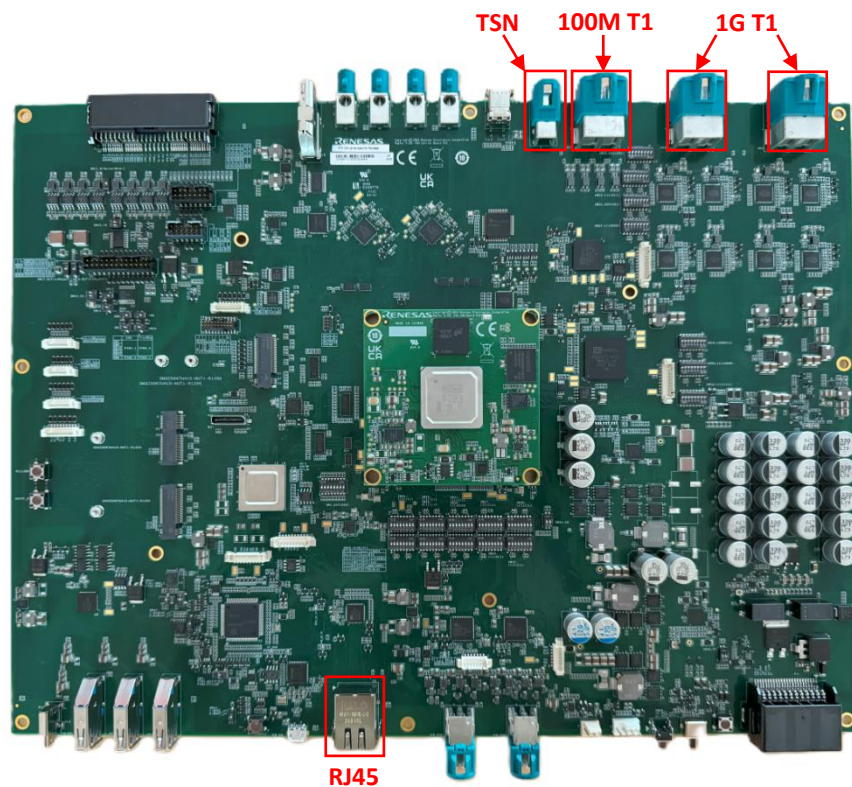


Figure 2. The position of each Ethernet connector

Steps to ensure Ethernet can work properly

1. Confirm hardware Ethernet cables are connected correctly. If the connector type is T1, check the mode is master or slave.
2. Use “ping” to check if the network is working or not. If the “ping” is successful, it indicates that the network connection of this interface is working appropriately.

```
$ ping -l eth1 8.8.8.8
PING 8.8.8.8 (8.8.8.8): 56 data bytes
64 bytes from 8.8.8.8: seq=0 ttl=54 time=4.901 ms
64 bytes from 8.8.8.8: seq=1 ttl=54 time=7.872 ms
...
```

3. If the “ping” fails, use “ifconfig” or “ip addr” to check if the network interface has obtained the correct IP address.

```
$ ifconfig eth1
eth1      Link encap:Ethernet  HWaddr 00:E0:4C:68:02:03
          inet addr:10.0.0.67  Bcast:10.0.0.255  Mask:255.255.255.0
          ...

$ ip addr show eth1
5: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP
group default qlen 1000
    link/ether 00:e0:4c:68:02:03 brd ff:ff:ff:ff:ff:ff
    inet 10.0.0.67/24 brd 10.0.0.255 scope global dynamic eth1
    ...
```

4. If the IP address is not obtained, use “udhcpc” to request an IP address again.

```
$ udhcpc -i eth1
udhcpc: started, v1.31.1
udhcpc: sending discover
udhcpc: sending select for 10.0.0.67
udhcpc: lease of 10.0.0.67 obtained, lease time 7200
/etc/udhcpc.d/50default: Adding DNS 168.95.1.1
/etc/udhcpc.d/50default: Adding DNS 8.8.8.8
```

5. If the IP address is still not obtained, reboot the device and restart from step 2.
6. If the IP address is obtained, but still cannot ping. Use “route -n” to check the routing table. Verify if the default route (Default Gateway) is set correctly to allow access to external networks.

```
$ route -n
Kernel IP routing table
Destination      Gateway         Genmask        Flags   Metric Ref Use  Iface
0.0.0.0          10.0.0.254     0.0.0.0        UG      10     0   0   eth1
10.0.0.0         0.0.0.0        255.255.255.0  U       0     0   0   eth1
```

3. How to Run UMSD_MCLI

Introduction

Here is the revised version[2]. If needed, please contact the RETRONIX PM. Thank you.

MCLI program emulates one Telnet command for UMSD API verification. Each UMSD API has a command, it validates both chip and API functions. The program has been tested on V4H-Raptor and Linux PC (Ubuntu 20.04 LTS).

Environment

Hardware:

<V4H-Raptor/Linux PC> + <Ethernet cable> + <Marvell Ethernet Switch>

Software:

<Linux Telnet package> + <UMSD_MCLI>

Default bootstrap settings on Raptor-V4H for 88Q6113 and 88Q5040[1]

Check the pin configuration before launching up UMSD_MCLI. RMU_SEL[2:0] define hardware RMU port. CPU_EN define internal CPU enabled or disabled. ADDR[3:0] define the target device number and the addressing mode is single or multi.

Table 1. Default bootstrap settings on SW18 – SW21 for 88Q6113

		1 (OFF)	0 (ON)	
SW18	[1] RMU_SEL0			000: Port1 (SGMII)
	[2] RMU_SEL1			001: Port7 (xMII)
	[3] RMU_SEL2			010: Port8 (xMII)
				011: Port9 (XFI)
				111: RMU disabled (default)
	[4] P9_SMODE0			00: 1000BASE-X/SGMII
	[5] P9_SMODE1			01: 5GBASE-R
				10: USXGMII
				11: 10GBASE-R
	[6] P10_SMODE0			00: 1000BASE-X/SGMII
	[7] P10_SMODE1			01: 5GBASE-R
				10: USXGMII
				11: 10GBASE-R
		1 (OFF)	0 (ON)	
SW19	[1] P7_MODE0			0: PORT 7 DISABLED
				1: PORT 7 RGMII
	[2] P8_MODE0			0: PORT 8 DISABLED
				1: PORT 8 RGMII
	[3] P1_MODE			0: 100FX
				1: 1000BASE-X/SGMII
	[4] P2_MODE			0: 100FX
				1: 1000BASE-X/SGMII
	[5] P3_MODE			0: 100FX
				1: 1000BASE-X/SGMII
SW20	[6] C3_LED(P11_0)			00/01: P11 PCIe
	[7] P8_OUTEN(P11_1)			10: P11 Eth port in 1000BASE-X/SGMII
				11: P11 Eth port with 2500BASE-X
	[8] CPU_EN			0: CPU DISABLED
				1: CPU ENABLED (Default)
		1 (OFF)	0 (ON)	
SW20	[1] SMI_ADDR0			0: SINGLE CHIP
	[2] SMI_ADDR1			
	[3] SMI_ADDR2			
	[4] SMI_ADDR3			
	[5] ACTIVE0			0: PORT 1-6 DISABLED
				1: PORT 1-6 ENABLED
SW21	[6] ACTIVE1			0: PORT 7-10 DISABLED
				1: PORT 7-10 ENABLED
SW21	[1] TEST			01: DAP
	[2] TEST			10: BSCAN

Table 2. Default bootstrap settings on SW22 and SW23 for 88Q5040

		1 (OFF)	0 (ON)	
SW22	[1] P5_MODE0			0x0: MII MAC Mode(FD)
	[2] P5_MODE1			0x1: MII PHY
	[3] P5_MODE2			0x2: MII MAC
				0x3: RESV
				0x4: RMII PHY
				0x5: RMII MAC
				0x6: DISABLE
				0x7: RGMII (Default)
	[4] P5_VDDOS			0x0: P5_VDDO = 2.5V
				0x1: P5_VDDO = 3.3V
	[5] P6_MODE0			0x0: MII MAC Mode(FD)
	[6] P6_MODE1			0x1: MII PHY
	[7] P6_MODE2			0x2: MII MAC
				0x3: RESV
				0x4: RMII PHY
				0x5: RMII MAC
				0x6: DISABLE
				0x7: RGMII (Default)
	[8] P6_VDDOS			0x0: P6_VDDO = 2.5V
				0x1: P6_VDDO = 3.3V
SW23		1 (OFF)	0 (ON)	
	[1] ADDR0			0: SINGLE CHIP
	[2] ADDR1			
	[3] ADDR2			
	[4] ADDR3			
	[5] ACTIVE			0: PORT 1-6 DISABLED
				1: PORT 1-6 ENABLED
	[6] GPIO_VDDOS			0: GPIO_VDDO = 2.5V
				1: GPIO_VDDO = 3.3V
	[7] SPI_VDDOS			0: SPI_VDDO = 2.5V
				1: SPI_VDDO = 3.3V
	[8] CPU_EN			0: CPU DISABLED
				1: CPU ENABLED

Table 3. Default bootstrap settings on SW24 and SW25 for 88Q5040

		1 (OFF)	0 (ON)			
SW24	[1] RMU_SEL0			0x0: PORT 1 (100BT1)		
	[2] RMU_SEL1			0x1: PORT 4 (100BT1/BTX/SGMII)		
	[3] RMU_SEL2			0x2: PORT 5 (xMII)		
				0x3-0x6: RESV		
				0x7: RMU DISABLE (default)		
	[4] P1_MASTER			0: SLAVE		
				1: MASTER (default)		
	[5] P2_MASTER			0: SLAVE		
				1: MASTER (default)		
	[6] P3_MASTER			0: SLAVE		
				1: MASTER (default)		
	[7] P4_MASTER			0: SLAVE		
				1: MASTER (default)		
		1 (OFF)	0 (ON)		PORT 4	PORT 5
SW25	[1] P45_MODE0			0x0	SGMII/1000X	RGMII
	[2] P45_MODE1			0x1	100BASE-TX	100BASE-FX
	[3] P45_MODE2			0x2	100BASE-TX	SGMII/1000X
				0x3	100BASE-TX	RGMII
				0x4	100BASE-FX	RGMII
				0x5	100BASE-T1	100BASE-FX
				0x6	100BASE-T1	SGMII/1000X
				0x7 (default)	100BASE-T1	RGMII
	[4] JTAG_EN			0: NORMAL OPERATION		
				1: JTAG ENABLE		
	[5] SSPI_EN			0: SLAVE SPI		
				1: SMI CPU		
	[6] SSPI_EN			0: TC10 ENABLE		
				1: TC10 DISABLE		
	[1] SSPI_MODE0			0x0: Polarity=0, Phase=0 (default)		
	[2] SSPI_MODE1			0x1: Polarity=0, Phase=1		
				0x2: Polarity=1, Phase=0		
				0x3: Polarity=1, Phase=1		

Procedure

This program can be run with an interactive interface or the configuration file. An interactive interface allows users to select different modes manually. If the mode is already determined, the configuration file can be used to bypass the manual input steps. The following are demonstrated in these two parts.

3.1. Run with an Interactive Interface

Start the program with an interactive interface.

```
$ UMSD_MCLI
```

Here is an example of running UMSD_MCLI through FW_RMU.

The following steps are:

1. Enter the Marvell Ethernet switch device ID.
2. Select the bus Interface. In this example, choose FW_RMU.
3. Input the target device number and Ether Type value in hexadecimal format.
4. Choose the NIC interface that connected to the switch.
5. Enter the telnet port to open the Telnet server.

```
Please enter the device (88Q5152, 88Q5072, 88Q6113, 88E6393X ...)
```

```
88Q6113
```

```
Interface List:
```

- 0 : SMI Interface
- 1 : SMI Multi-Chip
- 2 : RMU Interface
- 3 : FW RMU Interface
- 4 : Fake Device

```
Please select one interface: 3
```

```
Please enter the Target Device number and Ether Type value
```

```
(hex, Trg_Dev=0x0 and Etype=0x9101 please enter 0x0 0x9101 for example):
```

```
0x0 0x9101
```

```
===== NIC CARD List =====
```

```
(1) (null)
```

```
eth0
```

```
(2) (null)
```

```
eth1
```

```
(3) (null)
```

```
eth2
```

```
...
```

```
(14) (null)
```

```
can7
```

```
(15) (null)
```

```
tsn0
Please select NIC Index: 3
Using Interface: eth2

Over the Max Semaphore numbers
Spruce Driver Load Success

Please enter the telnet port(8888 for example):
8888
Listening on port 8888 for telnet
```

Here is another example of running UMSD_MCLI through SMI interface.

The steps are similar:

1. Enter the switch device ID.
2. Select the bus Interface. In this example, choose SMI Interface.
3. Select one SMI protocol.
4. Choose the NIC interface that connected to the switch.
5. Enter the telnet port to open the Telnet server.

Chapter 4 will introduce how to connect to Marvell Ethernet switch and how to use.

```
Please enter the device (88Q5152, 88Q5072, 88Q6113, 88E6393X ...)
88Q6113
Interface List:
    0 : SMI Interface
    1 : SMI Multi-Chip
    2 : RMU Interface
    3 : FW RMU Interface
    4 : Fake Device
Please select one interface: 0
SMI Protocol select:
    0: MDIO(Claue22)
    1: XMDIO(Claue45), only support by Oak/Spruce/Fir...
Please select one protocol: 1

===== NIC CARD List =====
(1) (null)
eth0
```


(2) (null)

eth1

(3) (null)

eth2

...

(14) (null)

can7

(15) (null)

tsn0

Please select NIC Index: 3

Over the Max Semaphore numbers

Spruce Driver Load Success

Please enter the telnet port(8888 for example):

8888

Listening on port 8888 for telnet

3.2. Run with the Configuration File

Start the program with the configuration file.

```
$ UMSD_MCLI -f umsd.cfg
```

The following is an example of executing UMSD_MCLI through FW_RMU after entering the above command.

```
===== NIC CARD List =====
```

```
(1) (null)
```

```
    eth0
```

```
(2) (null)
```

```
    eth1
```

```
(3) (null)
```

```
    eth2
```

```
...
```

```
(14) (null)
```

```
    can7
```

```
(15) (null)
```

```
    tsn0
```

```
Using Interface: eth2
```

```
Over the Max Semaphore numbers
```

```
Spruce Driver Load Success
```

```
Listening on port 8888 for telnet
```

To launch the program by the configuration file, it is necessary to verify that is correct. Here is the content of the configuration file below. The following steps are:

1. Confirm the switch device ID. In this case, inputDeviceId is 88Q6113.
2. Select the bus interface. In this example, choose FW_RMU.
3. Set the Telnet port.
4. Verify the parameters within FW_RMU.
 - a 、 dev_num depends on ADDR[3:0].
 - b 、 nic_name should be the interface connected to the switch.

Now, it is available to run the program using the configuration file.

```
#This file is the configure file for MCLI bus interface setting
```

```
bus_config {  
    #Choose one of the bus  
interface(SMI_Single/SMI_MultiChip/RMU/FW_RMU/Fake)  
    inputDeviceId 88Q6113  
    interface FW_RMU  
    telnet_port 8888  
  
    SMI_Single {  
        ...  
    }  
  
    SMI_MultiChip {  
        ...  
    }  
  
    RMU {  
        ...  
    }  
  
    FW_RMU {  
        #FW RMU mode (EtherType_DSA)  
        rmu_mode EtherType_DSA  
  
        #Device number (0 - 31)  
        dev_num 0  
    }
```

```
#Ether Type value(used by EtherType_DSA mode)
ether_type 0x9101

#NIC card name (e.g. eth0 for linux, "Intel(R) Ethernet Connection I217-LM"
for windows)
#nic_name "Intel(R) Ethernet Connection I217-LM"
nic_name eth2
}

Fake {
    ...
}
}
```

Here is another example of executing UMSD_MCLI through SMI interface. Before starting the program, make sure to check the content of the configuration file below. Chapter 4 will introduce how to connect to Marvell Ethernet switch and how to use.

```
#This file is the configure file for MCLI bus interface setting

bus_config {
    #Choose one of the bus
    interface(SMI_Single/SMI_MultiChip/RMU/FW_RMU/Fake)
    inputDeviceId 88Q6113
    interface SMI_Single
    telnet_port 8888

    SMI_Single {
        #Protocol(MDIO or XMDIO), only Amethyst/Oak/Spruce support the
        XMDIO protocol to access register
        smi_protocol XMDIO

        #NIC card name (e.g. eth0 for linux, "Intel(R) Ethernet Connection I217-LM"
        for windows)
        #nic_name "Intel(R) Ethernet Connection I217-LM"
        nic_name eth2
    }
}
```

```
SMI_MultiChip {  
    ...  
}  
  
RMU {  
    ...  
}  
FW_RMU {  
    ...  
}  
Fake {  
    ...  
}  
}
```

4. How to Use UMSD_MCLI

Use the command below as the Telnet client to connect to the Telnet server.

```
$ telnet [host ip] [port]
Trying [host ip]...
Connected to [host ip].
Escape character is '^'.
UMSD_MCLI test environment
```

After entering the environment, input 'help' can show available commands.

```
UMSD_MCLI> help

Commands available:
  help                Show available commands
  exit                Exit from current program
  history             Show a list of previously run commands
  arp                 Show tcam arp commands
  atu                 Show atu commands
  ...
  trunk              Show link aggregation (LAG/port trunking) commands
  umsd-api            Show umsd-api commands
  vlan                Show vtu commands
```

To know all the commands about vlan, input 'vlan help' would display all vlan command options.

```
UMSD_MCLI> valn help
vlan help [subcmd]: Display vtu command options
vlan addEntry [options] : Add vlan entry
vlan delAll : Flush all vlan entry
vlan delEntry <vlanId>: Delete the specified vlan entry
vlan dump : Find all the vlan entries and print them out
...
vlan setMode <modeValue>: Set the vlan mode
```

To read all the registers of the specified port, input the command 'port dump <port>'.

```
UMSD_MCLI> port dump 8
```

```
-----  
Port Status Register(0x0)          1a07  
Physical Control Register(0x1)      0003  
Flow Control Register(0x2)          0000  
Switch Identifier Register(0x3)      0002  
Port Control0 Register(0x4)         037f  
Port Control1 Register(0x5)         0000  
Port Based VLAN Map(0x6)            106ff  
...  
Queue Counter Registers(0x1b)       8000  
Port Queue Control Register(0x1c)   0000  
Port Queue Control2 Register(0x1d)  0000  
Cut Through Register(0x1e)          0000  
Debug Counter(0x1f)                 00f0
```

5. REFERENCE

[1] RRC-V4H_CB_V10_20240604_SCH_F.pdf

[2] UMSD_Debug_Tool_v7.0.0